

SimpleImputer & ML Workflow

A beginner-friendly study guide for Feature Engineering

1. What is SimpleImputer?

SimpleImputer is a tool from scikit-learn that automatically fills in missing values (NaN) in your dataset. Instead of dropping rows with missing data (which wastes information), it replaces them with a calculated value based on the strategy you choose.

Why do we need it?

Most machine learning models cannot handle missing values. If your dataset has NaN entries, the model will crash or produce wrong results. SimpleImputer solves this in one clean step.

Available Strategies

Strategy	Fills with...	Best used when...
mean	Average of the column	Data is normally distributed, no outliers
median	Middle value of the column	Data has outliers (more robust)
most_frequent	Most common value	Discrete/categorical-like numbers
constant	A fixed value you provide	Missing means 'unknown' or zero

Basic Code Example

```
from sklearn.impute import SimpleImputer
import numpy as np

# Sample data with missing values
X = [[25, 50000], [30, np.nan], [np.nan, 72000], [22, 45000]]

# Create the imputer
imputer = SimpleImputer(strategy='mean')

# Fit and transform
X_imputed = imputer.fit_transform(X)
print(X_imputed)
```

■ Note: fit() learns the mean/median from the data. transform() applies it. fit_transform() does both at once — use only on training data!

2. The ML Workflow (End-to-End)

Every machine learning project follows a standard workflow. SimpleImputer lives in Step 3 (Preprocessing). Here is the full picture:

#	Stage	What happens here
Step 1	Collect Data	Gather raw data from CSV, database, API, etc.
Step 2	Explore Data (EDA)	Check shape, data types, missing values, distributions.
Step 3	Preprocess / Feature Engineering	Handle missing values (SimpleImputer), encode categories, scale numbers.
Step 4	Split Data	Divide into training set and test set (e.g., 80/20 split).
Step 5	Train the Model	Feed the training data into a model (e.g., LinearRegression, RandomForest).
Step 6	Evaluate	Check performance on test set using metrics (accuracy, RMSE, etc.).
Step 7	Tune & Improve	Adjust hyperparameters, try different models or features.
Step 8	Deploy	Save the model and use it on new, real-world data.

3. SimpleImputer Inside the Workflow

The most important rule when using SimpleImputer in a real ML project is: fit on training data only, then transform both train and test separately.

```
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# Step 4 – Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# Step 3 – Impute (fit ONLY on train!)
imputer = SimpleImputer(strategy='median')
X_train = imputer.fit_transform(X_train) # learn + apply
X_test = imputer.transform(X_test) # apply only (no leakage)

# Step 5 – Train
model = LinearRegression()
model.fit(X_train, y_train)

# Step 6 – Evaluate
score = model.score(X_test, y_test)
print(f'R2 Score: {score:.2f}')
```

■ Why not fit on test data? Because the imputer would 'peek' at test statistics (mean/median) and use them during training — this inflates performance scores artificially. Always treat test data as if it's brand new, unseen data.

4. Pro Tip: Use a Pipeline

scikit-learn's Pipeline chains preprocessing steps and the model together. It automatically handles the fit/transform separation for you — no leakage risk!

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

pipe = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
    ('model', LinearRegression())
])

pipe.fit(X_train, y_train) # handles everything automatically
pipe.score(X_test, y_test)
```

5. Quick Reference Cheat Sheet

Task	Code
Import	<code>from sklearn.impute import SimpleImputer</code>
Mean strategy	<code>SimpleImputer(strategy='mean')</code>
Median strategy	<code>SimpleImputer(strategy='median')</code>
Constant fill	<code>SimpleImputer(strategy='constant', fill_value=0)</code>
Fit + transform	<code>imputer.fit_transform(X_train)</code>
Transform only	<code>imputer.transform(X_test)</code>
Check learned means	<code>imputer.statistics_</code>